


<https://cstaleem.com>


Doubly Linked List Insertion

Insertion in a **Doubly Linked List** (<https://cstaleem.com/introduction-to-doubly-linked-list-in-c>) (DLL) refers to adding a new node to the list. Inserting a node in the Doubly Linked List contains very simple steps in C. A **Doubly Linked List (DLL)** is a type of linked list where each node includes three components:

1. **Data** – to store the value.
2. **Pointer to the next node** (next) – links to the next node in the list.
3. **Pointer to the previous node** (prev) – links to the previous node in the list.

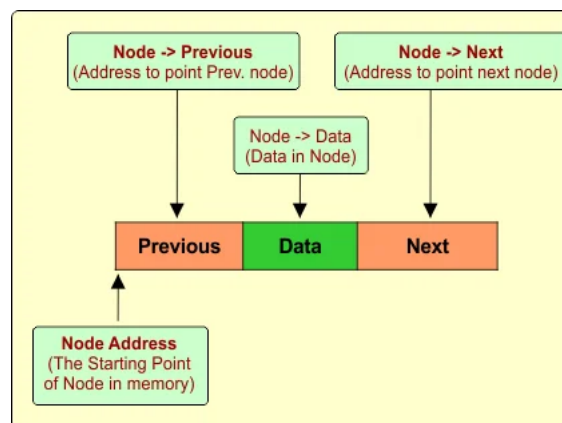
There are **different types of insertions** based on where the new node is added in the list:

- Insertion at the Beginning
- Insertion at the End
- Insertion After a Given Node
- Insertion Before a Given Node

Requirement For Insertion in Doubly Linked List

Before proceeding for the types of insertion in DLL, consider the following requirements for insertion in a doubly linked list [Master Coding Skills](#)

01: Consider a Node Structure



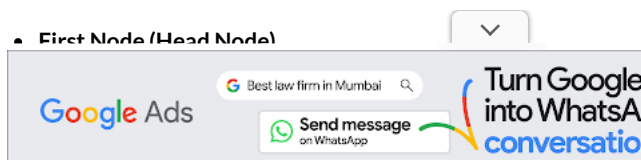
Node Structure: Doubly Linked List

```
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};
```

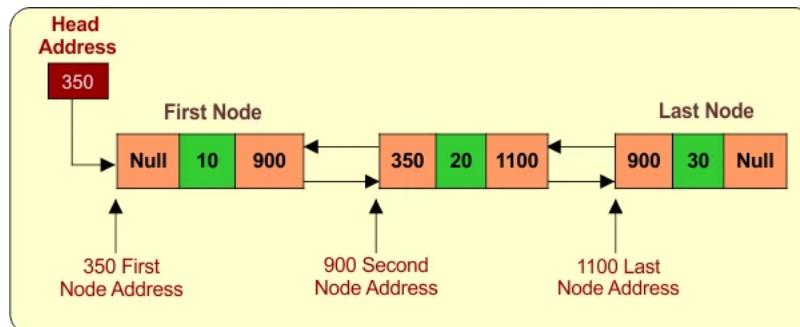
- data: Holds the value of the node.
- next: Pointer to the next node in the list.
- prev: Pointer to the previous node in the list. This allows us to traverse both forward and backward in the doubly linked list.

02: Consider a Doubly Linked List

Consider three nodes DLL where Head pointing to first node and it is also present somewhere in main memory. [Master Coding Skills](#)



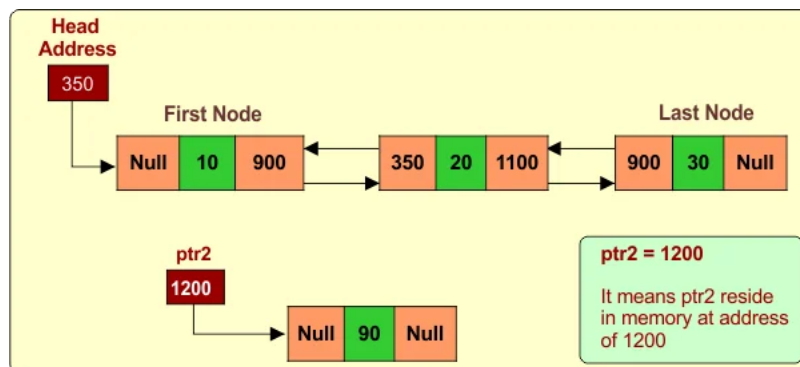
- Previous Pointer: NULL
- Second Node
 - Data: 20
 - Next Pointer: 1100 (Address of the third node)
 - Previous Pointer: 350 (Address of the first node)
- Third Node
 - Data: 30
 - Next Pointer: NULL
 - Previous Pointer: 900 (Address of the second node)



Consider a Doubly Linked List

03: Create a New Node for Insertion

The createNode function dynamically creates a new node and initializes its values Master Coding Skills



Create new Node in Linked list

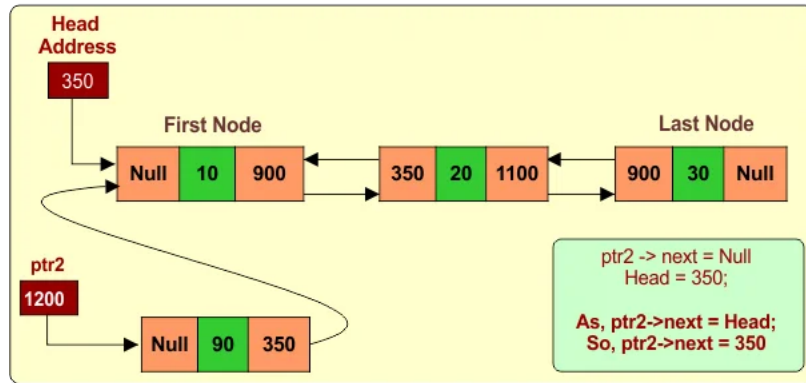
```
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory allocation failed\n");
        exit(1);
    }
    newNode->data = data;
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}
```

- A new node is allocated in memory using malloc.
- The data is initialized with the input value.
- Both next and prev pointers are set to NULL since the node is initially isolated.

Type 01: Insertion at Beginning in DLL

Step 01: ptr2->next = Head;

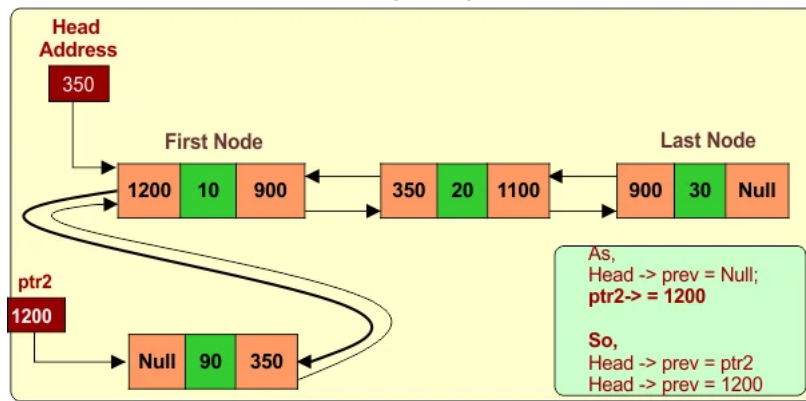
The statement (ptr2->next = head;) updates the next pointer of the new node (ptr2) to point to the current head of the linked list. Master Coding Skills

ptr2->next = Head;

Insert Node at the beginning of doubly Linked List - Step 01

Step 02: Head->prev = ptr2;

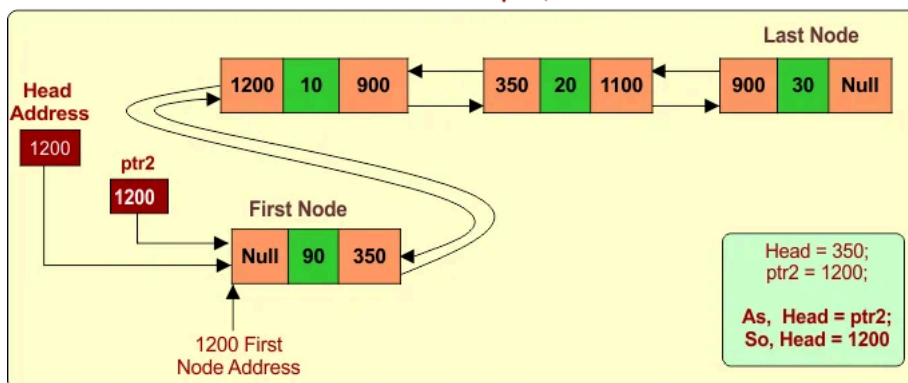
The statement head->prev = ptr2 updates the prev pointer of the current head node to point to the new node (ptr2) being inserted at the beginning of the doubly linked list.

Head->prev = ptr2;

Create new Node in Linked list

Step 03: Head = ptr2;

- After the assignment Head = ptr2;, both Head and ptr2 point to the same memory location (the node). Any changes made to the node through Head or ptr2 will affect the same node since they share the same memory address.

Head = ptr2;

Insert Node at the beginning of doubly Linked List

C Program: Insertion at the beginning in DLL

```
#include <stdio.h>
#include <stdlib.h> // Define the structure of a node for a doubly linked list
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
}; // Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory allocation failed\n");
        exit(1);
    }
}
```

```

newNode->data = data;
newNode->next = NULL;
newNode->prev = NULL;
return newNode;
} // Function to insert a node at the beginning of the doubly linked list
struct Node* insertAtBeginning(struct Node* head, int data) {
    struct Node* newNode = createNode(data); // Point the new node's next to the current head
    newNode->next = head;

    // Update the previous pointer of the current head (if it exists)
    if (head != NULL) {
        head->prev = newNode;
    }

    // The new node becomes the new head
    return newNode;
}  Master Coding Skills

// Function to display the doubly linked list
void displayList(struct Node* head) {
    struct Node* ptr2 = head;
    while (ptr2 != NULL) {
        printf("%d <-> ", ptr2->data);
        ptr2 = ptr2->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    // Manually creating the doubly linked list
    struct Node* head = createNode(10);
    head->next = createNode(20);
    head->next->prev = head;
    head->next->next = createNode(30);
    head->next->next->prev = head->next;
    head->next->next->next = createNode(40);
    head->next->next->next->prev = head->next->next;

    printf("Initial doubly linked list: ");
    displayList(head);

    // Insert a new node with value 90 at the beginning
    int newData = 90;

    printf("\nInserting %d at the beginning...\n", newData);
    head = insertAtBeginning(head, newData);

    printf("Doubly linked list after insertion: ");
    displayList(head);

    return 0;
}

```

Output:

Initial doubly linked list: 10 <-> 20 <-> 30 <-> 40 <-> NULL

Inserting 90 at the beginning...

Doubly linked list after insertion: 90 <-> 10 <-> 20 <-> 30 <-> 40 <-> NULL

Type 02: Insertion at End in DLL

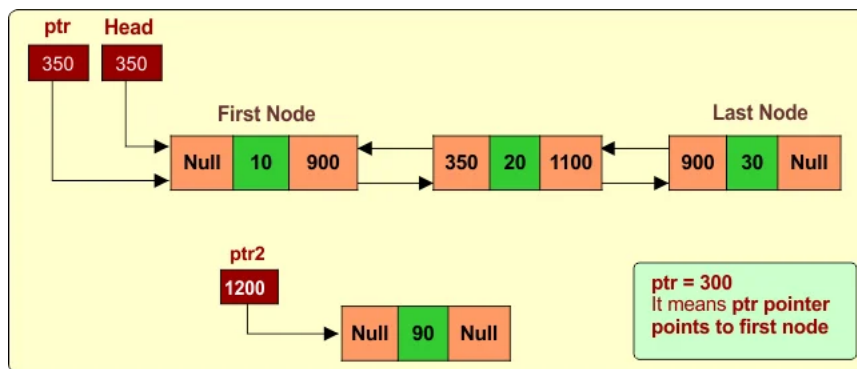
step 01: (ptr = head);

ptr is a temporary pointer used to navigate through the nodes of the linked list without modifying the **head** pointer. The **head** pointer always points to the first node of the list. Modifying **head** would lose track of the beginning of the list.  Master Coding Skills

The line `ptr = head;` ensures that:

- Traversal begins at the start of the list.
- The head pointer remains unchanged.



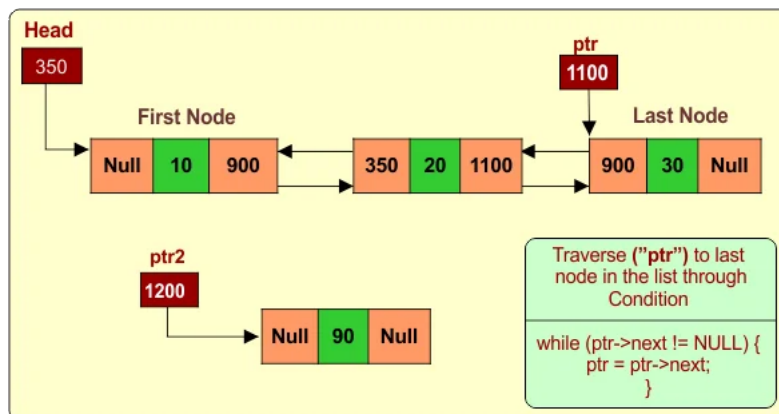


Create new head pointer ("ptr") in Linked list

step 02: ptr traversal Condition

```
while (ptr->next != NULL) {
    ptr = ptr->next;
}
```

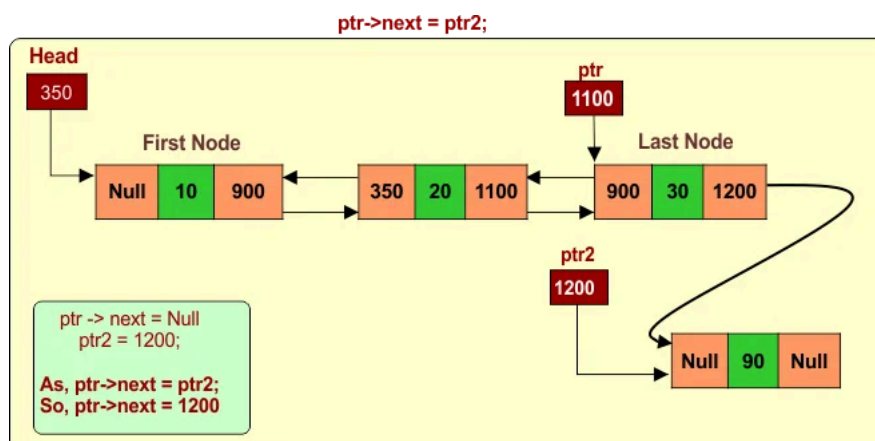
This loop is used to traverse the linked list from the current node (ptr) until it reaches the last node in the list. The last node is identified by the condition `ptr->next == NULL`. [Learn Coding Online](#)



Traverse ("ptr") to Last Node

step 03: ptr->next = ptr2

As the condition is false in step 02, statement `(ptr->next = ptr2)` is executed. By setting `ptr->next = ptr2`, the next pointer of the last node now points to the new node (ptr2).

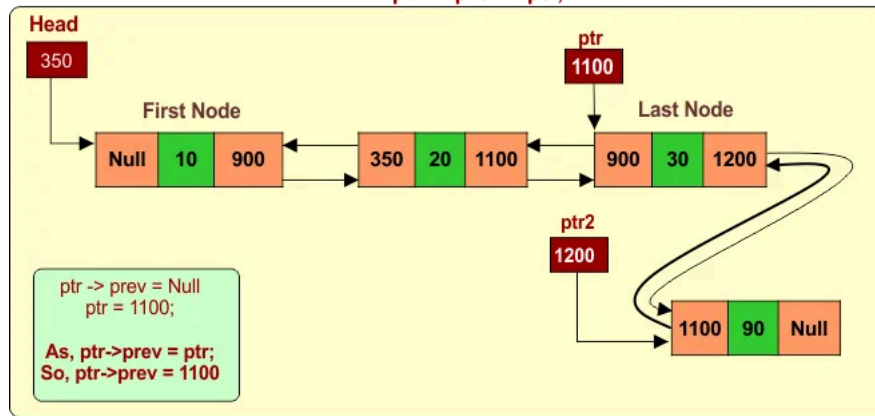


Insert Node at the end of Linked List

step 04: (ptr2->prev = ptr);

The line `ptr2->prev = ptr` ensures that the new node (ptr2) is properly backward-linked to the previous last node (ptr), completing the bidirectional linkage necessary in a doubly linked list.

[Master Coding Skills](#)

ptr2->prev = ptr;**Insert Node at the end of Linked List**

C Program: Insertion at the End in Doubly Linked List

Discover more

Data >

Learn C++ Coding >

Master Coding Skills >

```
ptr2->prev = NULL;
return ptr2;
}
```

◀ <https://cstaleem.com/insertion-at-the-end-of-linked-list-between-singly-linked-list-and-doubly-linked-list>

```
struct Node* insertAtEnd(struct Node* head, int data) {
    struct Node* ptr2 = createNode(data);
```

Master Coding Skills

Intro to Data Structure	the new node becomes the head if (head == NULL) { return ptr2; }	+
Linked List		-
Delete a Node in Linked List	// Traverse to the last node while (ptr->next != NULL) { ptr = ptr->next; }	
Insert Node at Beginning of linked list in c		
Insert the Node at a Particular Position in Linked List	ptr->next = ptr2; ptr2->prev = ptr;	
Insert Node at End of Linked List In C	return head; // Head remains unchanged }	
Introduction to Doubly linked list in C	// Function to display the doubly linked list void displayList(struct Node* head) { struct Node* ptr = head; while (ptr != NULL) { printf("%d <-> ", ptr->data); ptr = ptr->next; }	
Difference Between Singly linked list and Doubly linked list	printf("NULL\n"); }	
Singly Linked List		
Advantages of Doubly Linked List Over Singly Linked List	// Main function int main() { // Creating the doubly linked list struct Node* head = createNode(10); head->next = createNode(20); head->next->prev = head; head->next->next = createNode(30); head->next->next->prev = head->next; head->next->next->next = createNode(40); head->next->next->next->prev = head->next->next;	
Circular Singly linked list in c	displayList(head);	
Difference Between Singly Linked List and Circular Linked List	// Insert a new node with value 100 at the end int endData = 90; printf("\nInserting %d at the end...\n", endData); insertAtEnd(head, endData); displayList(head);	
Advantages of Circular Linked List		
Applications of Circular Linked List		
Circular Linked List Program In C		

Tail Pointer in Linked List (<https://cstaleem.com/tail-pointer-in-linked-list>)

Initial doubly linked list: 10 <-> 20 <-> 30 <-> 40 <-> NULL	+
Inserting 90 at the end...	+
Doubly linked list after insertion at the end: 10 <-> 20 <-> 30 <-> 40 <-> 90 <-> NULL	+
Sorting Algorithm	+